



Capstone Project



Today's Lesson

Goal

Architecture

Detail requirements



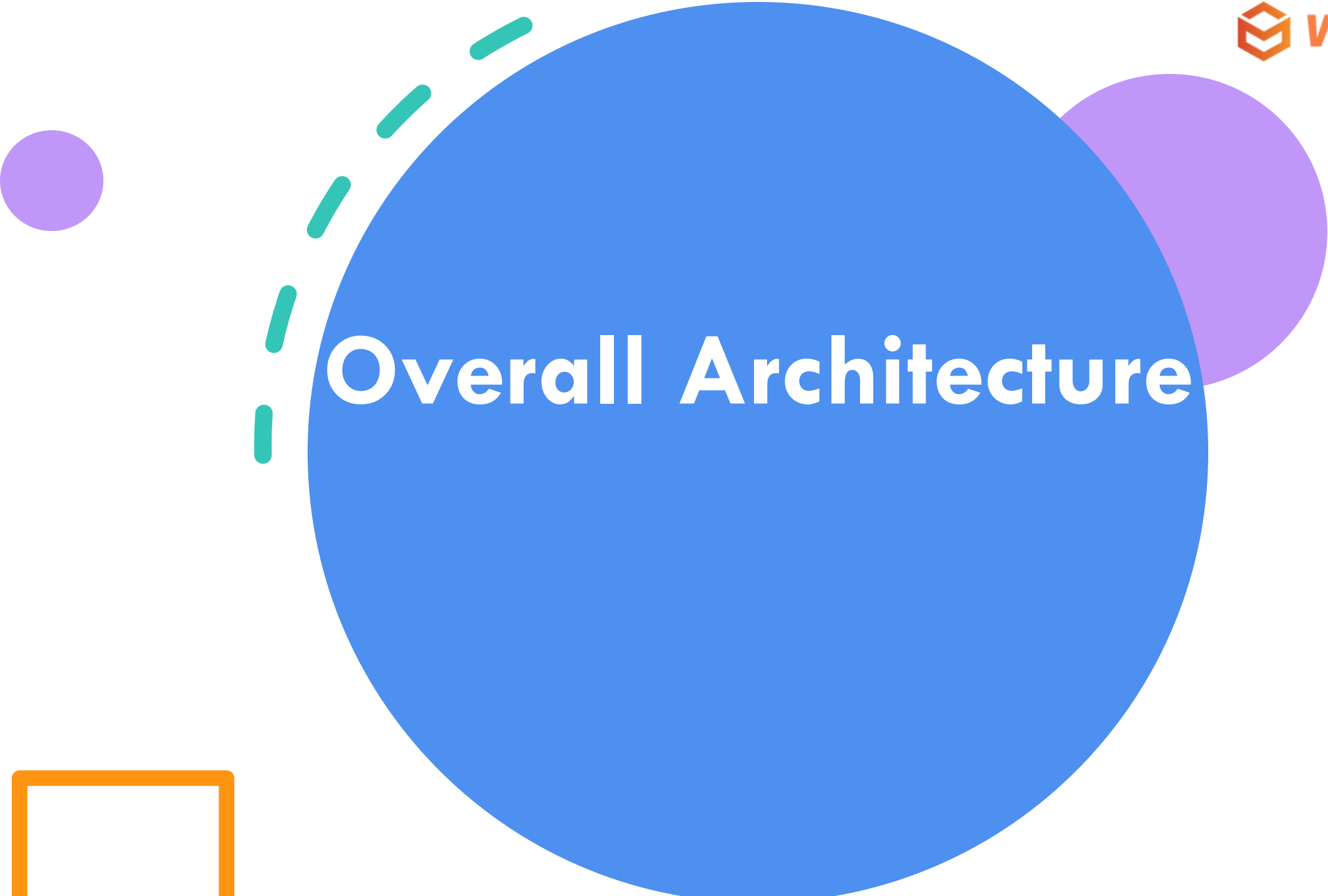
Goal

Goal

To implement:

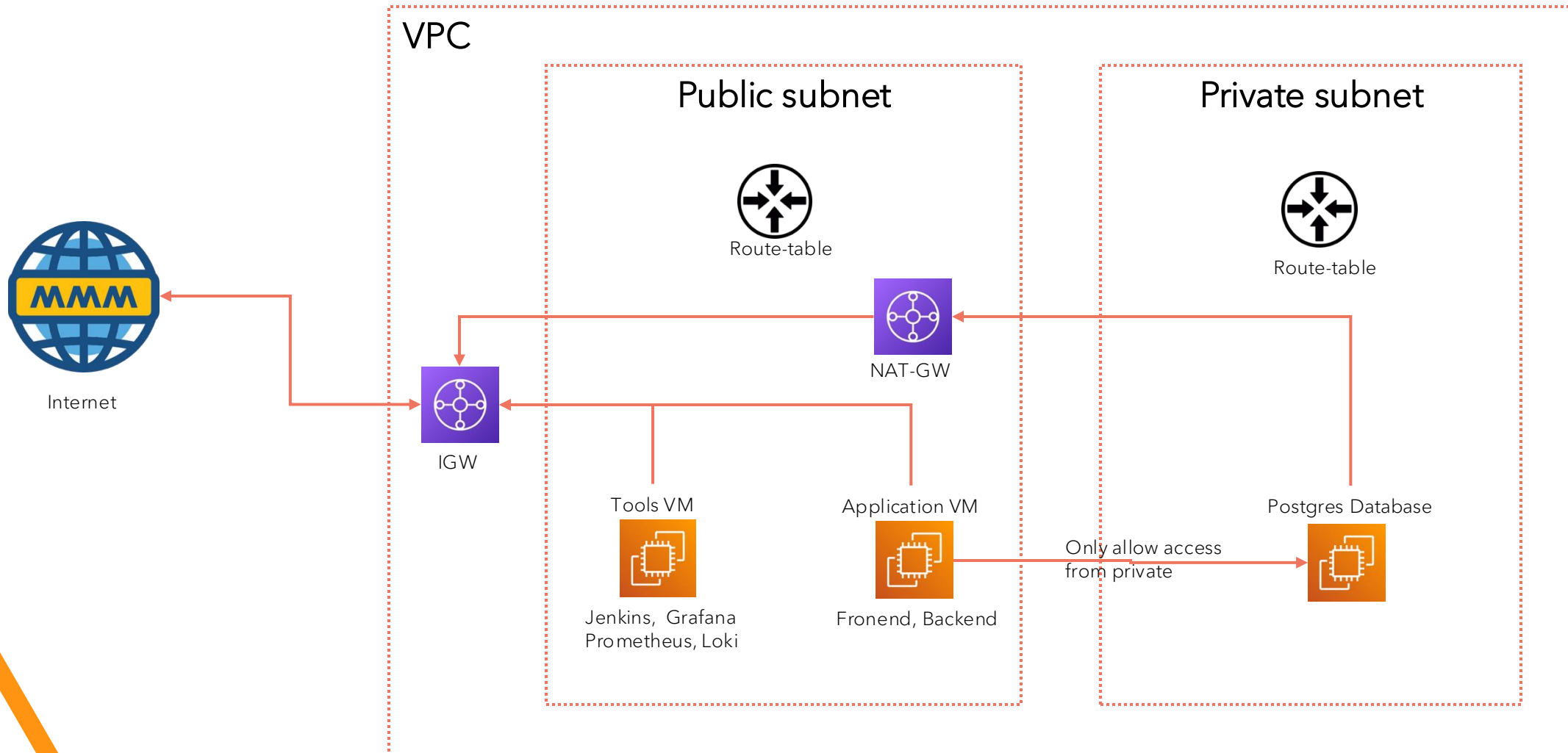
- A highly secured, reliable and scalable **infrastructure**
- A robust, automated **Continuous Integration/Continuous Delivery (CI/CD)** pipeline
- A comprehensive **monitoring solution**, leveraging a pre-configured AWS environment and a professional open-source DevOps toolchain.

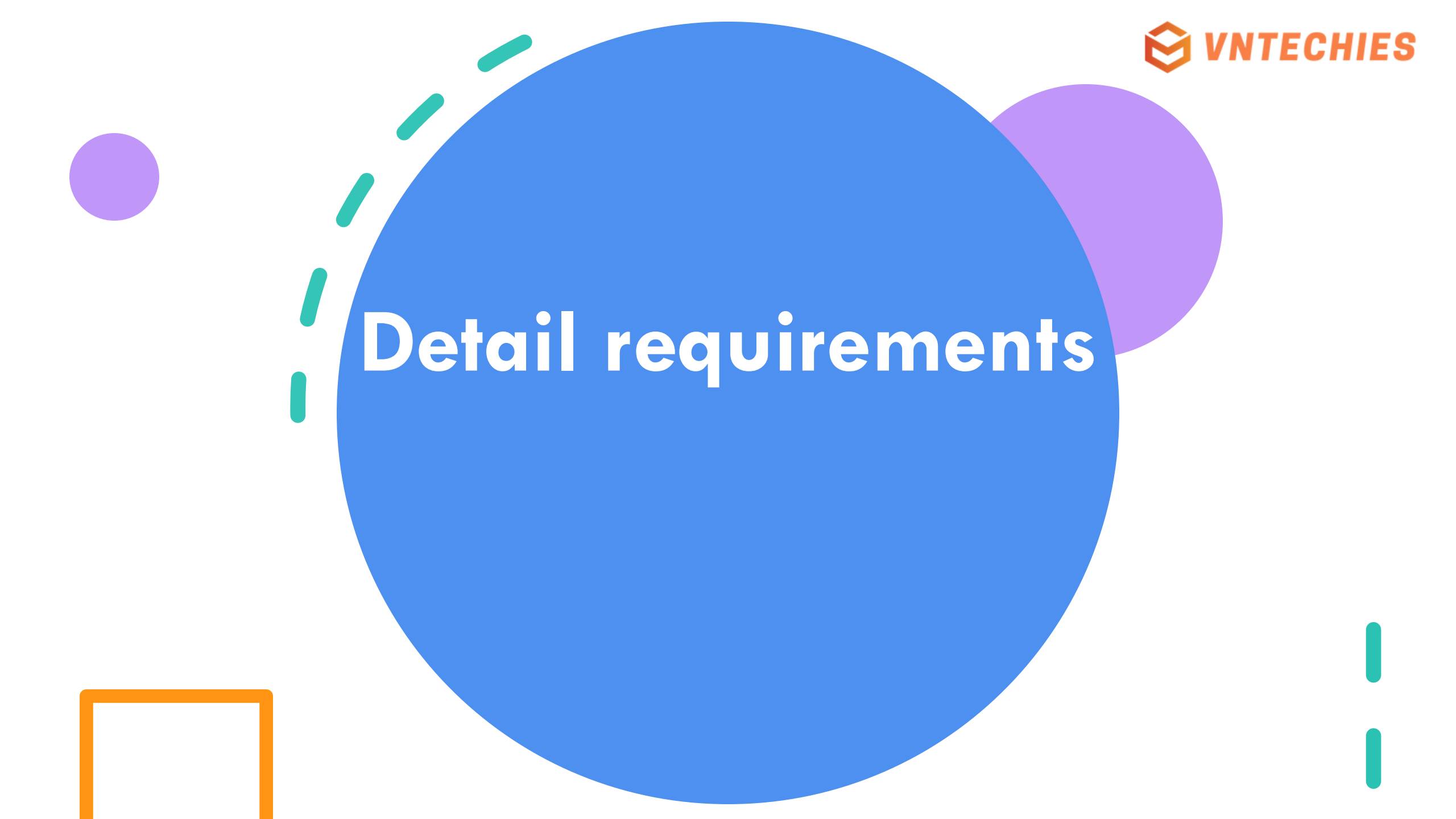
for a containerized 3-tier To-Do list application



Overall Architecture

Overall Architecture





Detail requirements

Application

Define and build production-ready **Docker images** for the application components:

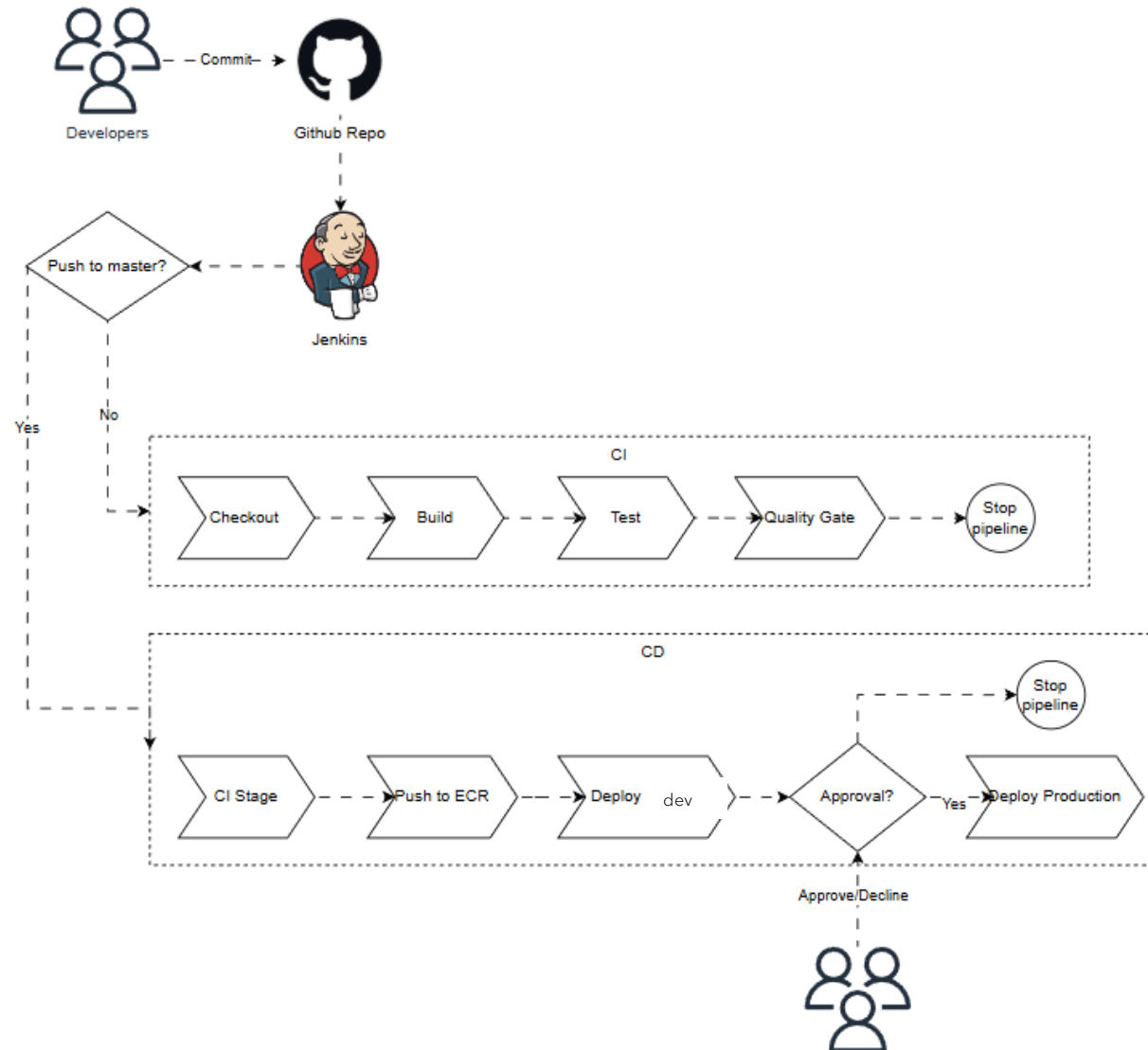
Layer	Repository	Technology	Requirement
Frontend	https://github.com/devopsway/devops-bootcamp-todolist-frontend	React	Create a minimal, multi-stage Dockerfile .
Backend API	https://github.com/devopsway/devops-bootcamp-todolist-backend-api	Node.js / Express	Create a minimal, multi-stage Dockerfile . Configure to accept DB connection details via environment variables.
Database	PostgreSQL (AWS RDS)	PostgreSQL	Connection details must be sourced securely by the Backend API deployments.

Infrastructure

The following resources need to be provisioned and operational in AWS:

- **Tools EC2:** A dedicated **AWS EC2 instance** running **Jenkins, Prometheus, Grafana, Loki**
- **Application EC2:** Running frontend, backend
 - For **Development(dev)** environment: Deploy application using docker, docker compose
 - For **Production(prod)** environment: Deploy application to k8s (using Kind: <https://kind.sigs.k8s.io/>)
- **Database EC2:** instance running **PostgreSQL**. (can be installed using docker)
 - Contain 2 database: one for **dev** one for **prod**
- **Image Registry:** Two operational Docker Hub repositories (one for Frontend, one for Backend).

CI/CD



Note: The entire pipeline must be defined using a dynamic **Jenkins Pipeline-as-Code** (Jenkinsfile) that adapts its execution based on the Git trigger.

Monitoring

A unified observability stack must be deployed, connected, and configured to monitor the health of the application and the deployment environments.

1. Metrics Collection (Prometheus)

- **Deployment:** Deploy Prometheus .
- **Scraping Targets:** Configure Prometheus to scrape metrics from:
 - **EC2 Host:** Resource utilization (CPU, Memory, Disk) via **Node Exporter**.

2. Log Aggregation (Loki)

- **Loki Deployment:** Deploy **Loki** alongside Prometheus.
- **Promtail Agents:** Deploy **Promtail** as a log shipper on:
 - The **EC2 instance** to ship Docker container logs.

Monitoring

3. Visualization and Dashboards (Grafana)

- **Deployment:** Deploy Grafana.
- **Data Sources:** Configure Grafana to use both **Prometheus** and **Loki** as data sources.
- **Dashboards:** Create a minimum of two custom dashboards:
 - **Infrastructure Dashboard:** Showing resource usage and EC2 health.
 - **Application Dashboard:** Showing API performance metrics (Latency, Error Rate, Throughput) alongside a **Loki Logs Panel** configured for **logs correlation** (filtering logs by time range and labels based on metric data).



Practice Time